STM32H750 电赛智能车 – 原理与实现

5.2 PID调节原理-运动精准控制的基础

InnoLegend 技术团队

目 录

CONTENTS

1 PID理论基础

2 PID系统结构

3 PID实现

第一部分

PID理论基础

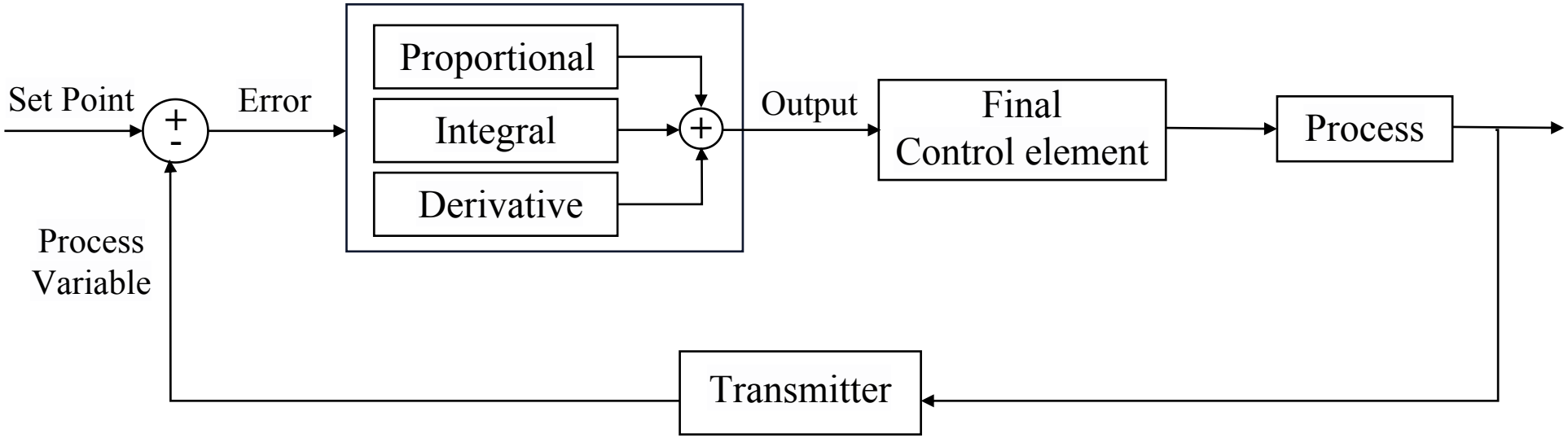
Introduction to Control Direction of Electronic Design Competition

1.1 PID原理

简单来说，PID控制器是一种用来自动调节系统的装置，采用闭环控制。它通过不断监测当前状态与期望目标之间的差， 根据这个差，通过**比例（Proportional，P）**、**积分（Integral，I）**、**微分（Derivative，D）**三个环节动态调整系统输出，以达到期望的目标。

https://zhuanlan.zhihu.com/p/448979690

1.1 PID原理



调节类型	调节操作	调节优势	调节劣势
Proportional(比例调节)	$u_p = k_P \cdot e(t)$	正比于相应速度，偏置越大，调节力度越大	产生一个无法消除的offset
Integral(积分调节)	$u_I = k_I \cdot \int_0^t e(t)dt$	可以消去offset这种系统静态误差	过大系统稳定性下降
Derivative(微分调节)	$u_D = k_D \cdot \frac{de(t)}{dt}$	可以对误差变化率预测和提前调整	噪声毛刺多，容易产生震荡

https://zhuanlan.zhihu.com/p/226114262

1.1 PID原理

InnoLegend

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}$$

调节类型	调节操作	调节优势	调节劣势
Proportional(比例调节)	$u_p = k_P \cdot e(t)$	正比于相应速度，偏置越大，调节力度越大	产生一个无法消除的offset
Integral(积分调节)	$u_I = k_I \cdot \int_0^t e(t)dt$	可以消去offset这种系统静态误差	过大系统稳定性下降
Derivative(微分调节)	$u_D = k_D \cdot \frac{de(t)}{dt}$	可以对误差变化率预测和提前调整	噪声毛刺多，容易产生震荡

https://zhuanlan.zhihu.com/p/226114262

1.2 PID原理理解

InnoLegend

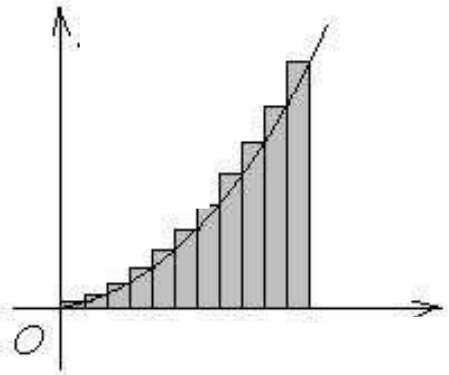
用PID给浴缸放水

- Proportional：看着水位差来调节水龙头。
- Integral：结合误差与时间调节水龙头
- Derivative： 预测而给水龙头“踩刹车”。

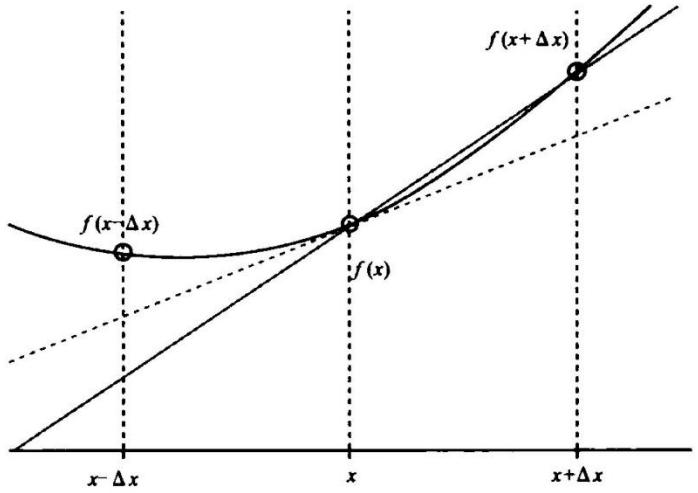
1.3 PID离散化

以固定的采样周期 T 对时间进行分割：

- 积分项离散化：使用矩形法离散化。 $$\int_0^t e(\tau)\,d\tau \approx T\sum_{i=0}^{k} e(i)$$



- 微分项离散化：用后向差分离散化。 $$\frac{de(t)}{dt} \approx \frac{e(k)-e(k-1)}{T}$$



1.4 PID离散化—位置式PID

InnoLegend

$$u_k = K_p \cdot e_k + K_i \sum_{j=0} e_j + K_d(e_k - e_{k-1})$$

其中：k——采样序号，k=0,1,2，……；

u_k——第 k 次采样时刻的计算机输出值；

e_k——第 k 次采样时刻输入的偏差值；

e_{k-1}——第 k-1 次采样时刻输入的偏差值；

Ki——积分系数，Ki=Kp*T/Ti；

Kd——微分系数，Kd=Kp*Td/T；

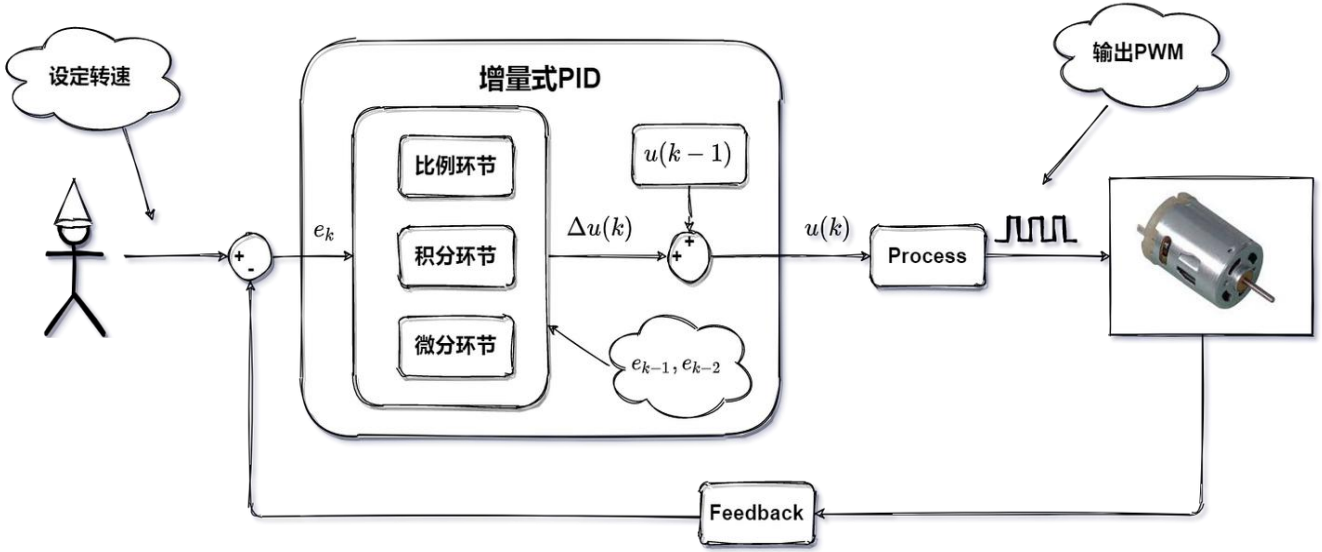
因为有误差积分一直累加，当前的输出与过去的所有状态都有关系。

1.5 PID离散化—增量式PID

InnoLegend

$$\Delta u(k) = k_P \cdot [e(k) - e(k-1)] + k_I \cdot e(k) + k_D \cdot [e(k) - 2e(k-1) + e(k-2)]$$

增量式PID输出控制量的变化（Δu），其本质是对位置式PID的差分，通过差分操作，消去积分项的累加，**仅依赖最近几次的误差**



图源：https://zhuanlan.zhihu.com/p/226114262

第二部分

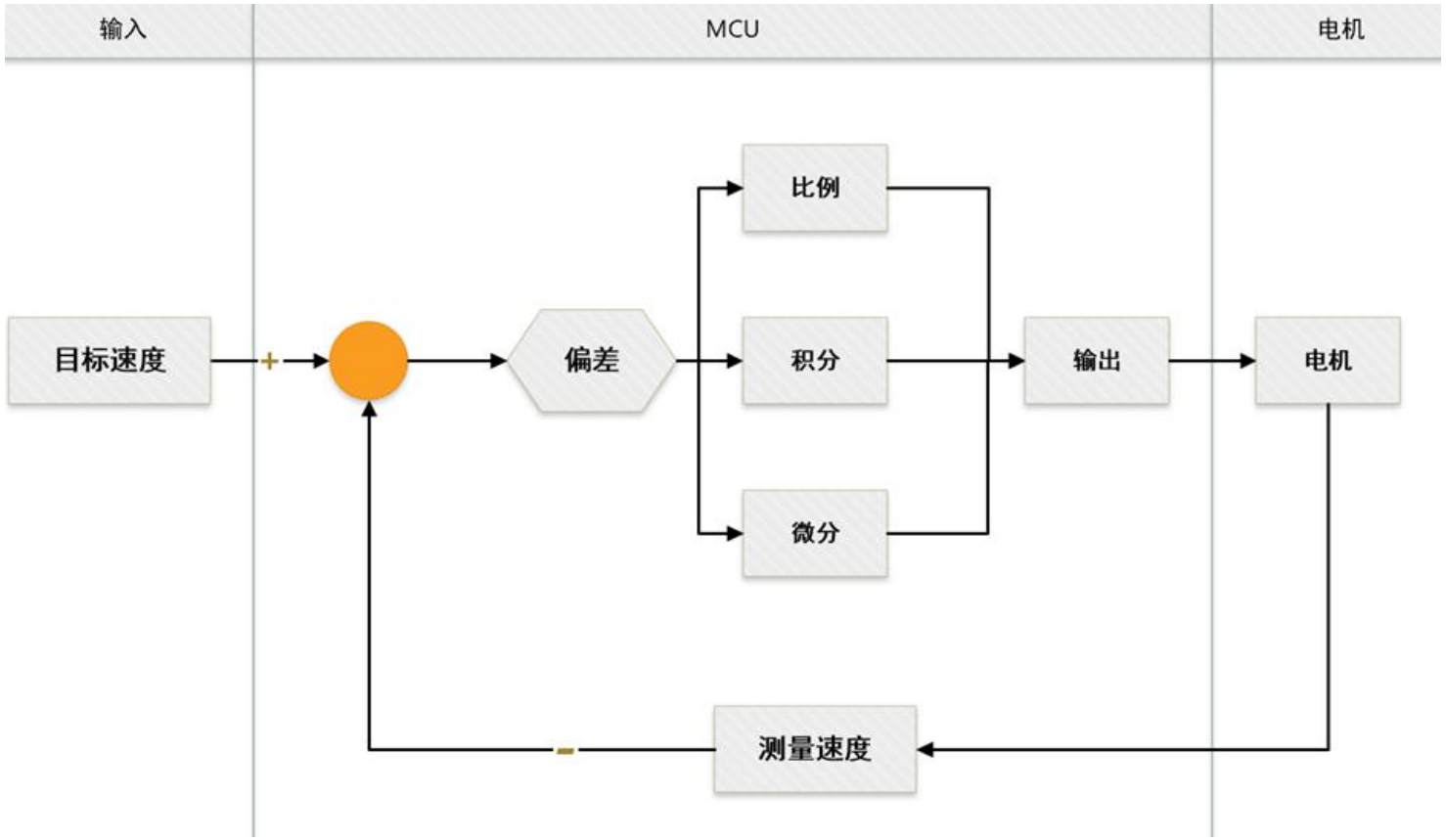
PID系统结构

How to quickly get started with smart cars

2.1 单级与串级PID

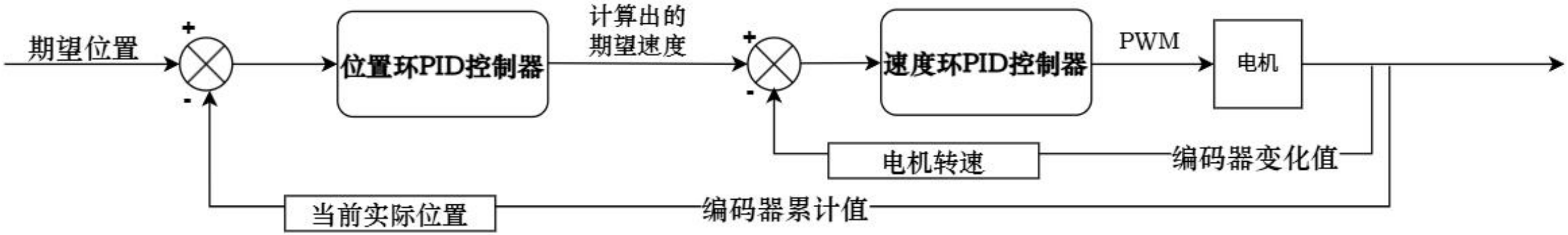
InnoLegend

单级PID（速度环）



2.2 串级PID

串级PID（外环位置环，内环速度环）



第三部分

PID实现

How to quickly get started with smart cars

3.1 PID代码实现

InnoLegend

$$\Delta u(k) = k_P \cdot [e(k) - e(k-1)] + k_I \cdot e(k) + k_D \cdot [e(k) - 2e(k-1) + e(k-2)]$$

3.2 PID调参

InnoLegend

通用方法（经验法）：

- 输入值为最大限幅的60%；Ki、Kd设为0，从0开始加Kp，当出现临界振荡时停止，再逐步回调Kp，直到振荡消失，记此时Kp值为Kp′，然后将Kp回调到Kp′的60%~70%。
- 从0逐步增大Ki，观察响应曲线，直到稳态误差消除，若出现周期性缓慢振荡且响应速度过慢，要缓慢调小Ki。
- 如果有过冲现象，从0逐步小步幅增大Kd，如果发现系统对噪声敏感，响应曲线高频抖动则需要回调Kd。

STM32H750 电赛智能车 – 原理与实现

欢迎各位同学交流、指正

InnoLegend 技术团队